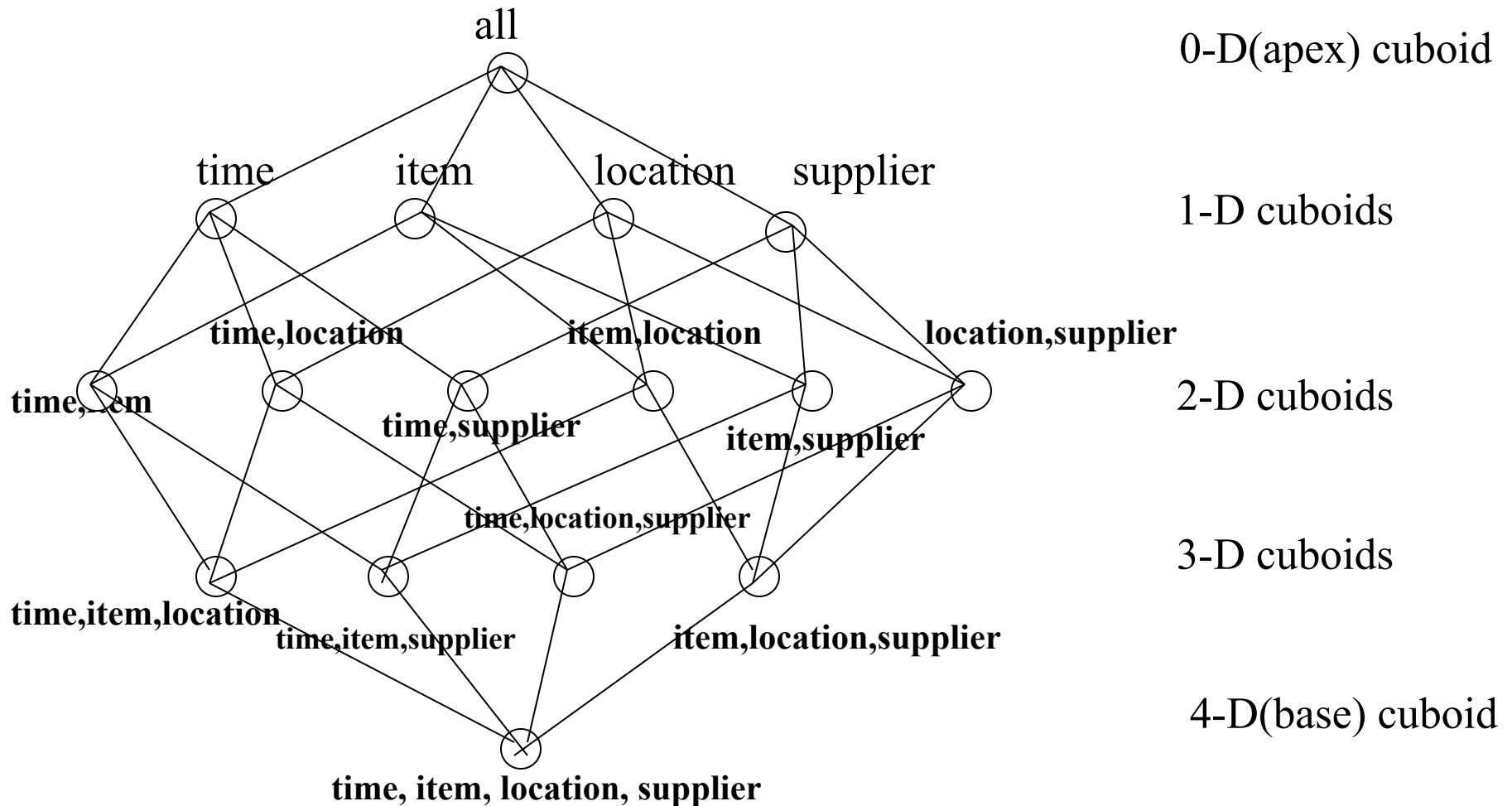

Data Cube Computation

Cube: A Lattice of Cuboids



Efficient Computation of Data Cubes

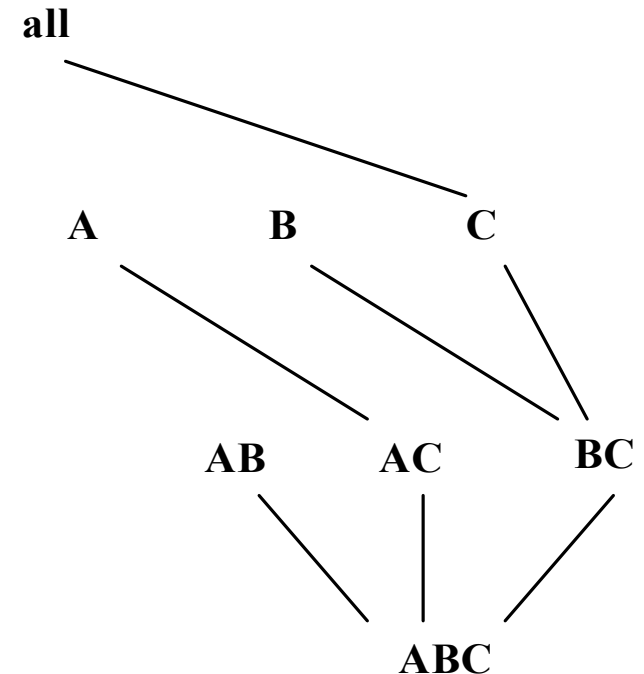
- Preliminary cube computation tricks (Agarwal et al'96)
- Computing full/iceberg cubes: 3 methodologies
 - Top-Down: **Multi-Way** array aggregation (Zhao, Deshpande & Naughton, SIGMOD'97)
 - Bottom-Up:
 - **Bottom-up** computation: BUC (Beyer & Ramakrishnan, SIGMOD'99)
 - H-cubing technique (Han, Pei, Dong & Wang: SIGMOD'01)
 - Integrating Top-Down and Bottom-Up:
 - Star-cubing algorithm (Xin, Han, Li & Wah: VLDB'03)
- High-dimensional OLAP: A Minimal Cubing Approach
- Computing alternative kinds of cubes:
 - Partial cube, closed cube, approximate cube, etc.

Preliminary Tricks (Agarwal et al. VLDB'96)

- Sorting, hashing, and grouping operations are applied to the dimension attributes in order to reorder and cluster related tuples
- Aggregates may be computed from previously computed aggregates, rather than from the base fact table
 - **Smallest-child:** computing a cuboid from the smallest, previously computed cuboid
 - **Cache-results:** caching results of a cuboid from which other cuboids are computed to reduce disk I/Os
 - **Amortize-scans:** computing as many as possible cuboids at the same time to amortize disk reads
 - **Share-sorts:** sharing sorting costs cross multiple cuboids when sort-based method is used
 - **Share-partitions:** sharing the partitioning cost across multiple cuboids when hash-based algorithms are used

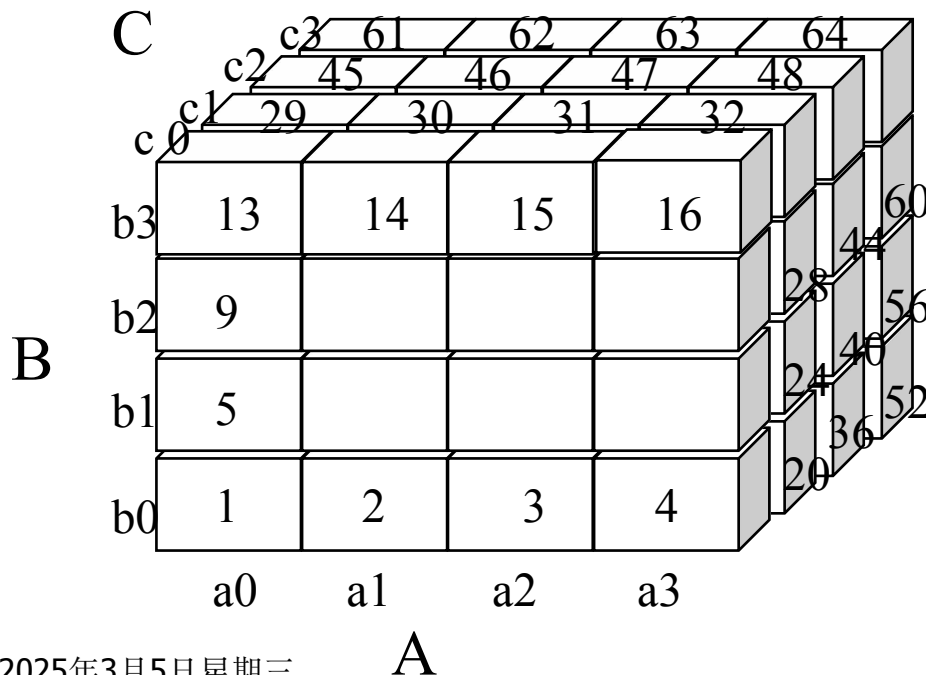
Multi-Way Array Aggregation

- Array-based “bottom-up” algorithm
- Using multi-dimensional chunks
- No direct tuple comparisons
- Simultaneous aggregation on multiple dimensions
- Intermediate aggregate values are re-used for computing ancestor cuboids
- Cannot do *Apriori* pruning: No iceberg optimization



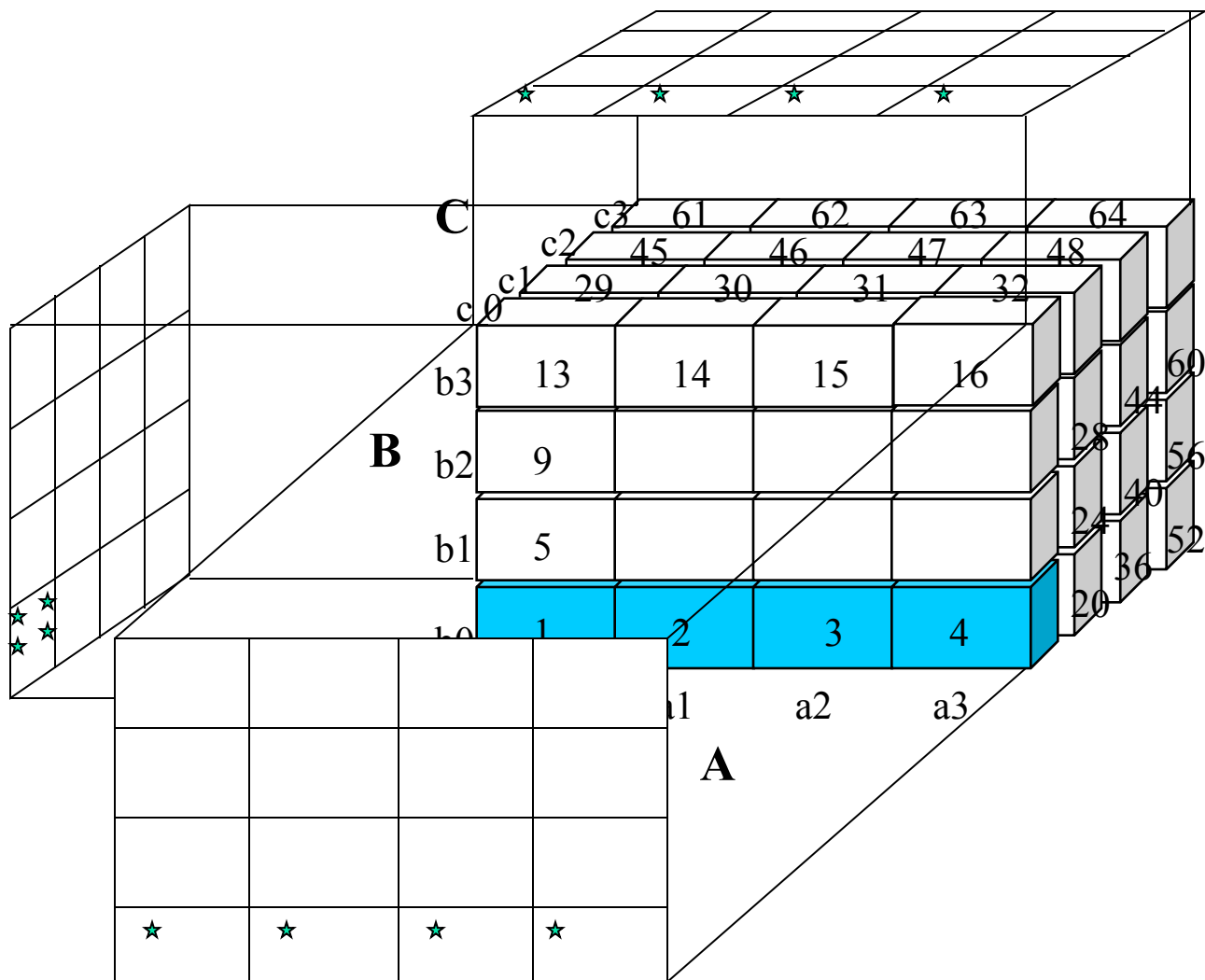
Multi-way Array Aggregation for Cube Computation (MOLAP)

- Partition arrays into chunks (a small subcube which fits in memory).
- Compressed sparse array addressing: (chunk_id, offset)
- Compute aggregates in “multiway” by visiting cube cells in the order which minimizes the # of times to visit each cell, and reduces memory access and storage cost.

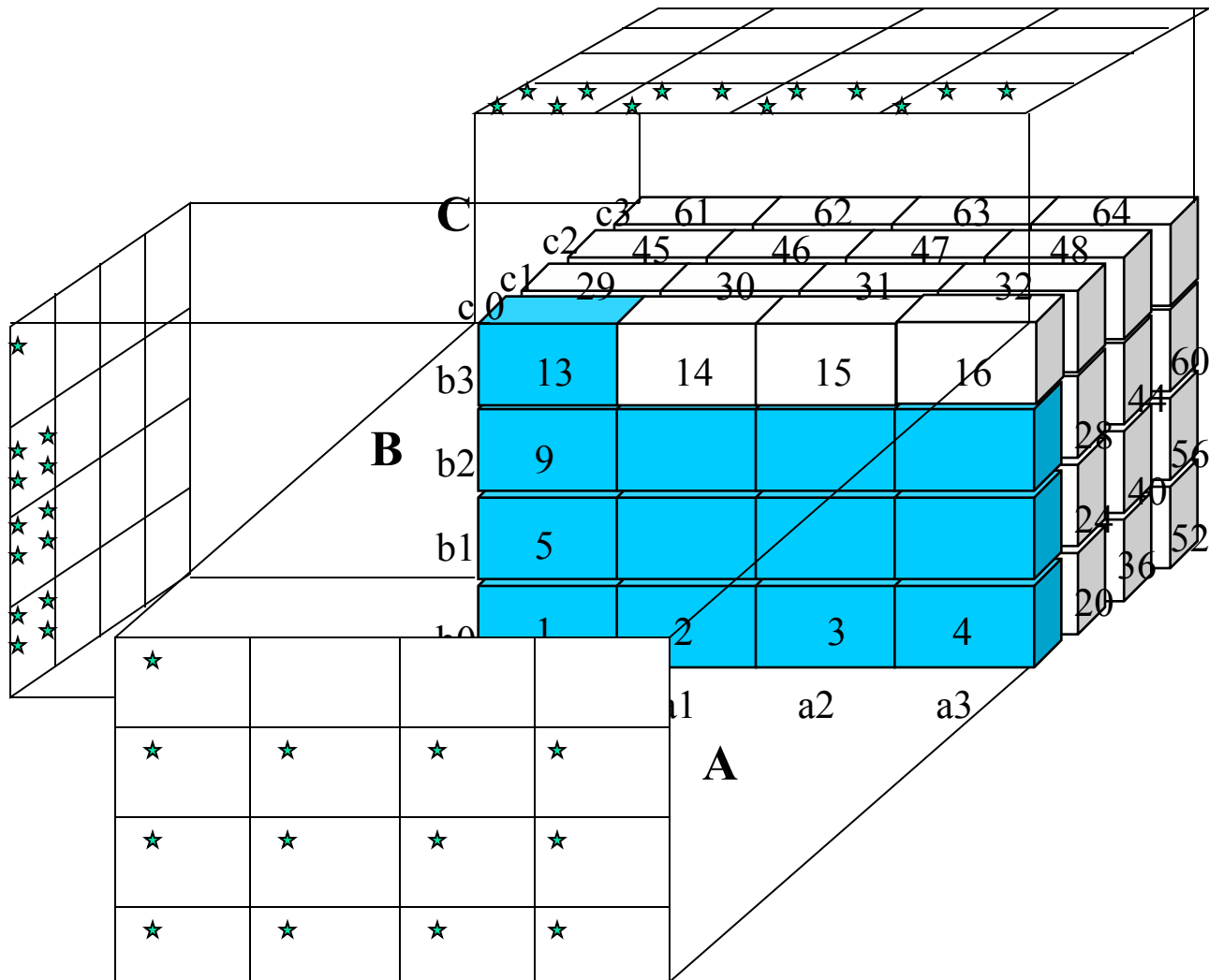


What is the best traversing order to do multi-way aggregation?

Multi-way Array Aggregation for Cube Computation



Multi-way Array Aggregation for Cube Computation

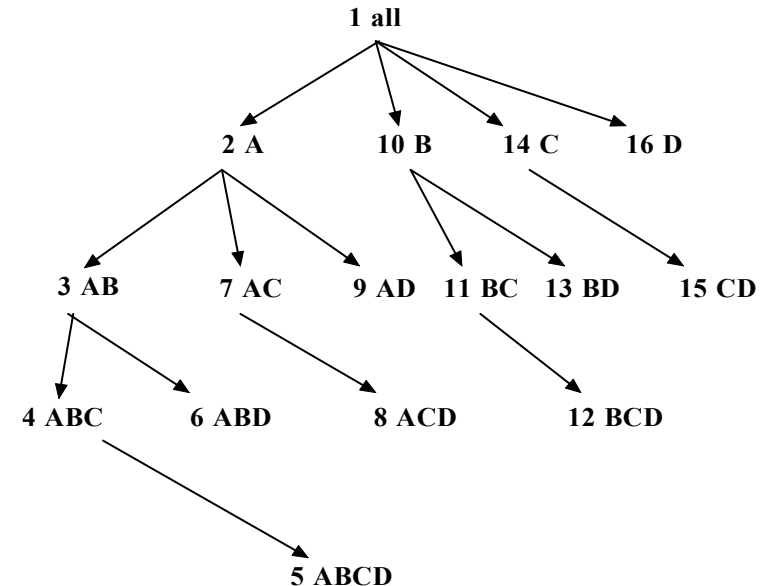
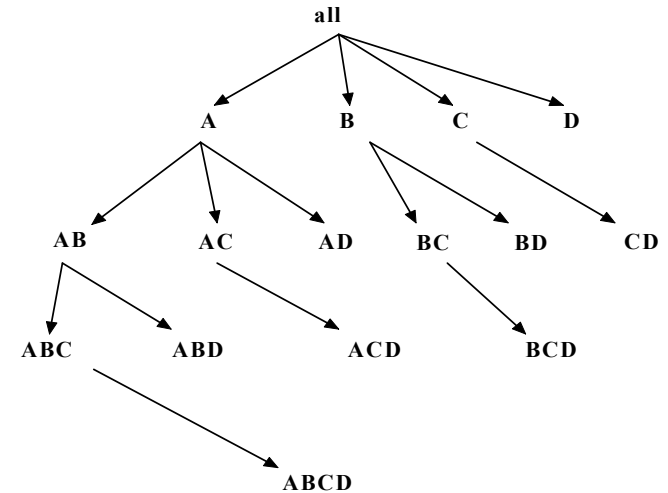


Multi-Way Array Aggregation for Cube Computation (Cont.)

- Method: the planes should be sorted and computed according to their size in ascending order
 - Idea: keep the smallest plane in the main memory, fetch and compute only one chunk at a time for the largest plane
- Limitation of the method: computing well only for a small number of dimensions
 - If there are a large number of dimensions, “top-down” computation and iceberg cube computation methods can be explored

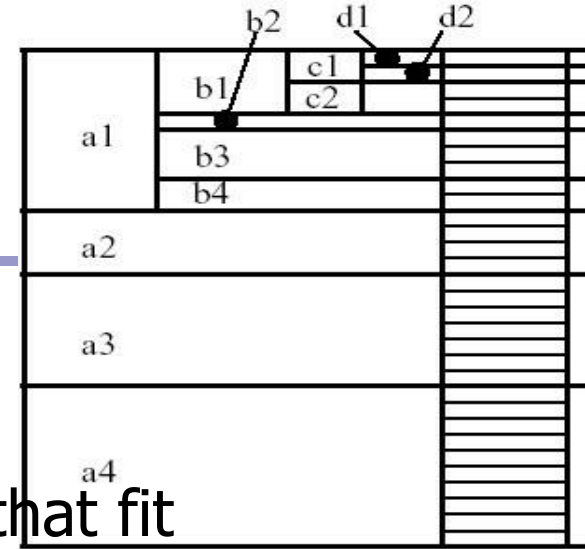
Bottom-Up Computation (BUC)

- BUC (Beyer & Ramakrishnan, SIGMOD'99)
- Bottom-up cube computation
(Note: top-down in our view!)
- Divides dimensions into partitions and facilitates iceberg pruning
 - If a partition does not satisfy *min_sup*, its descendants can be pruned
 - If *minsup* = 1 $\Rightarrow$ compute full CUBE!
- No simultaneous aggregation



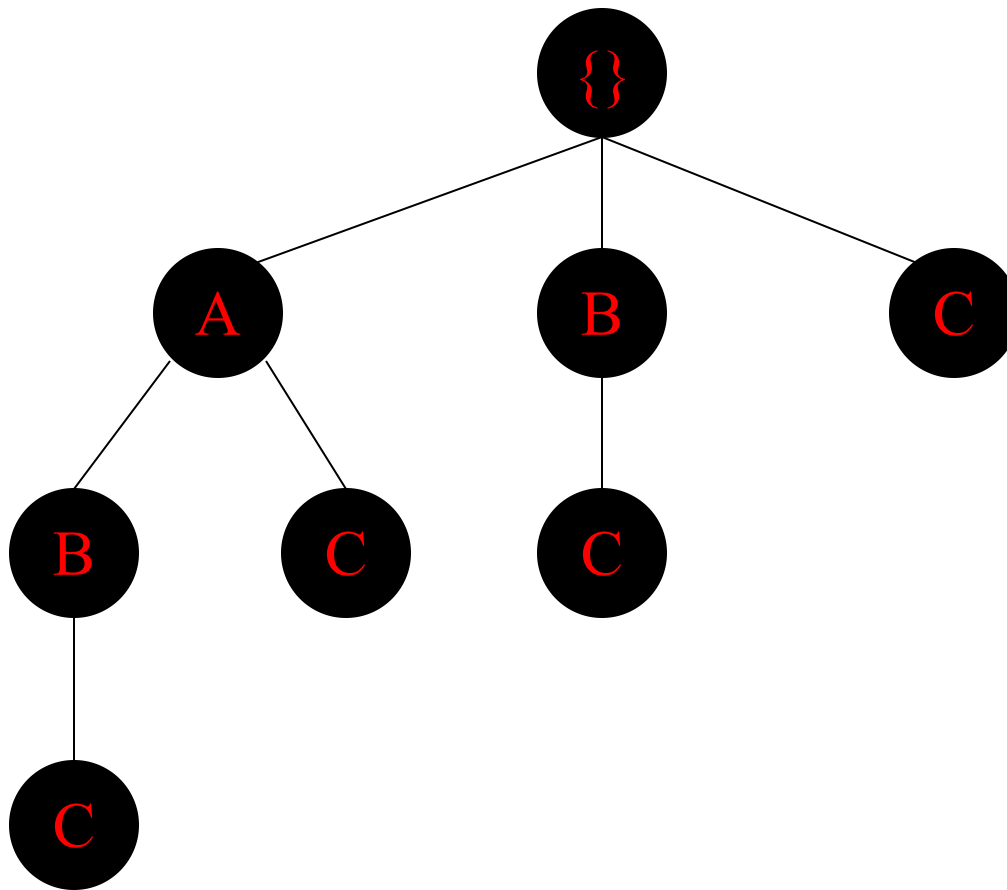
BUC: Partitioning

- Usually, entire data set can't fit in main memory
- Sort *distinct* values, partition into blocks that fit
- Continue processing
- Optimizations
 - Partitioning
 - External Sorting, Hashing, Counting Sort
 - Ordering dimensions to encourage pruning
 - Cardinality, Skew, Correlation
 - Collapsing duplicates
 - Can't do holistic aggregates anymore!



```
Procedure BottomUpCube(input, dim)
1: Aggregate(input); //Places result in outputRec
2: if input.count() == 1 then //Optimization
    WriteAncestors(input[0],dim); return;
3: write outputRec;
4: for d = dim; d < numDims; d++ do
5:   let C = cardinality[d];
6:   Partition(input, d, C, dataCount[d]);
7:   let k = 0;
8:   for i = 0; i < C; i++ do //for each partition
9:     let c = dataCount[d][i]
10:    outputRec.dim[d] = input[k].dim[d];
11:    BottomUpCube(input[k...k+c],d+1);
12:    k += c;
13:   end for
14:   outputRec.dim[d] = ALL;
15: end for
```

BUC Algorithm



{}

A

AB

ABC

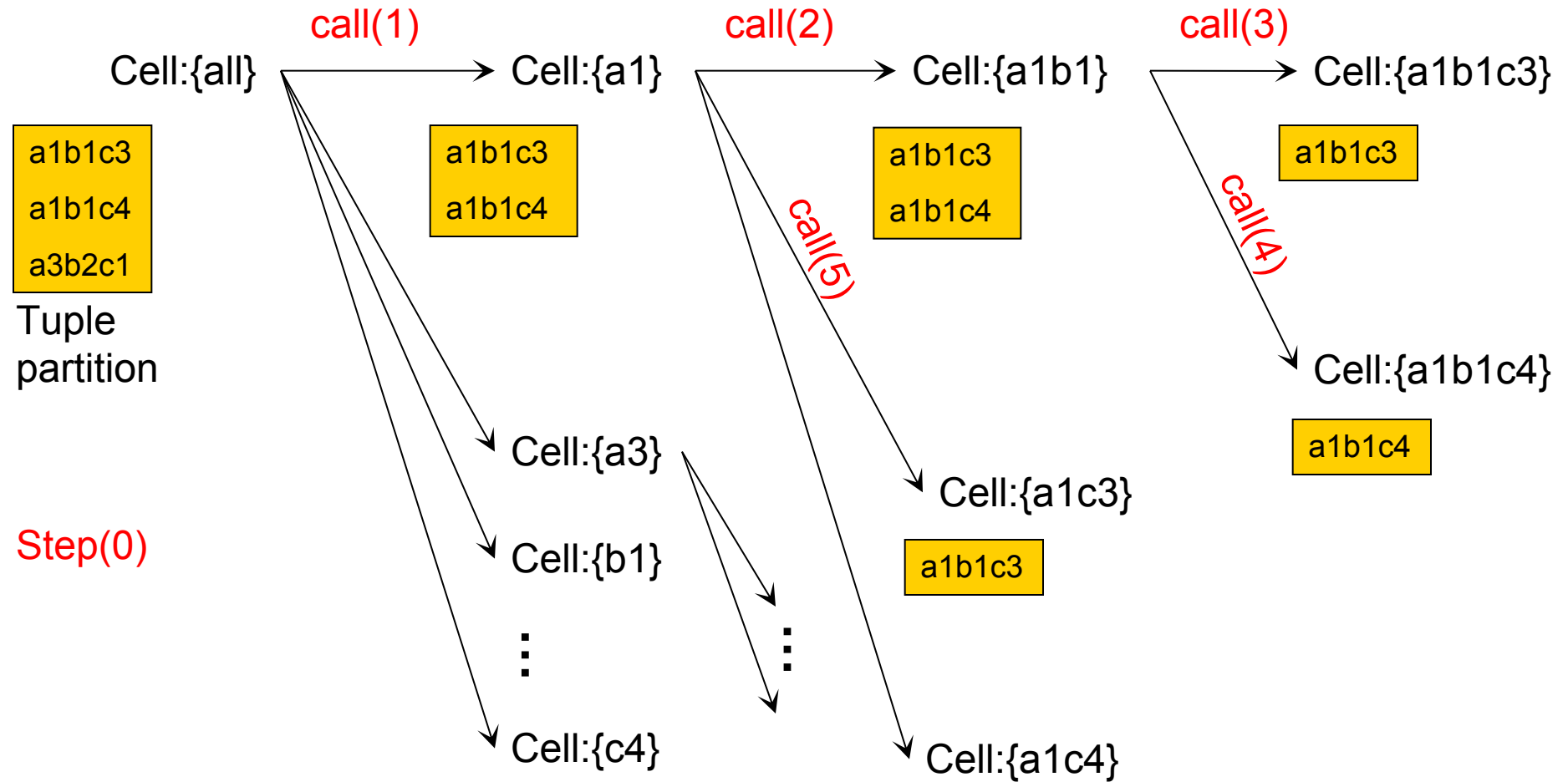
AC

B

BC

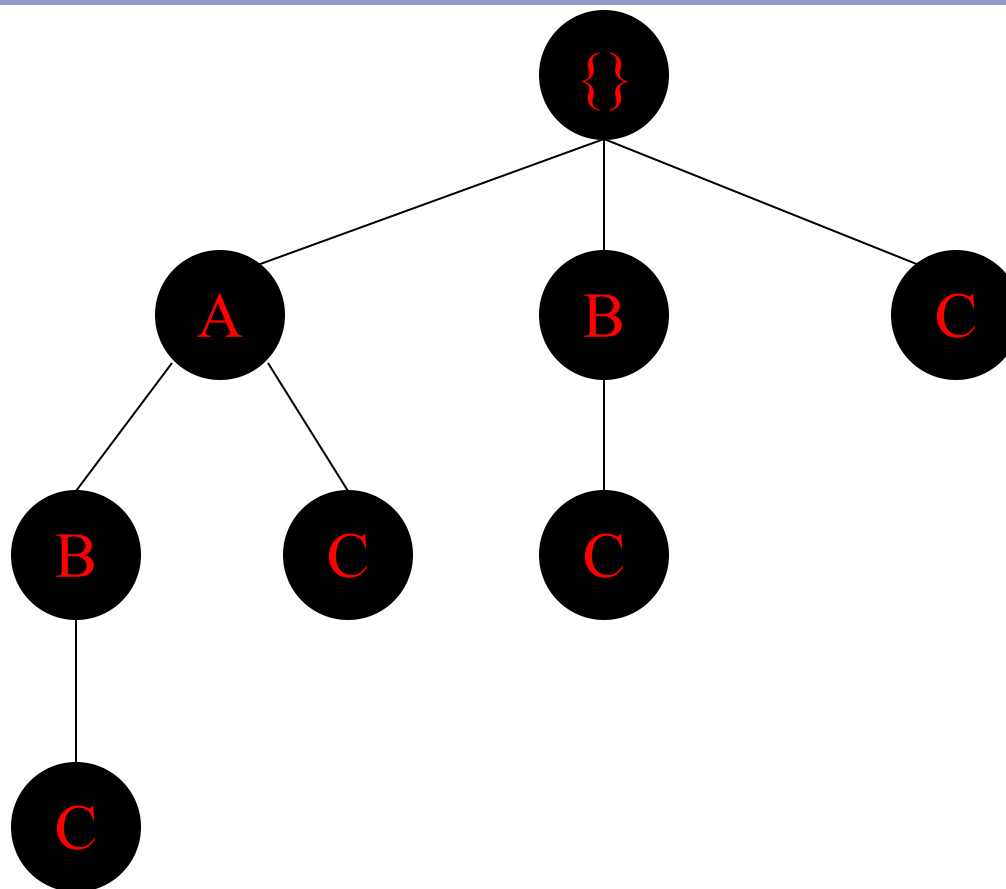
C

Example



BUC Algorithm

T	S	P	Sales
1	1	1	10
1	1	2	20
1	2	1	30
1	2	2	40
2	1	1	50
2	1	2	60
2	2	1	70
2	2	2	80



{}

A

AB

ABC

AC

B

BC

C

序号	T	S	P	Sales	序号	T	S	P	Sales
1	All	All	All	360	16	2	2	1	70
2	1	All	All	100	17	2	2	2	80
3	1	1	All	30	18	2	All	1	120
4	1	1	1	10	19	2	All	2	140
5	1	1	2	20	20	All	1	All	140
6	1	2	All	70	21	All	1	1	60
7	1	2	1	30	22	All	1	2	80
8	1	2	2	40	23	All	2	All	220
9	1	All	1	40	24	All	2	1	100
10	1	All	2	60	25	All	2	2	120
11	2	All	All	260	26	All	All	1	160
12	2	1	All	110	27	All	All	2	200
13	2	1	1	50					
14	2	1	2	60					
15	2	2	All	150					

BUC Algorithm

- 适合稀疏数据
- 可以共享排序
- 当划分只有一个元组时不需要继续划分

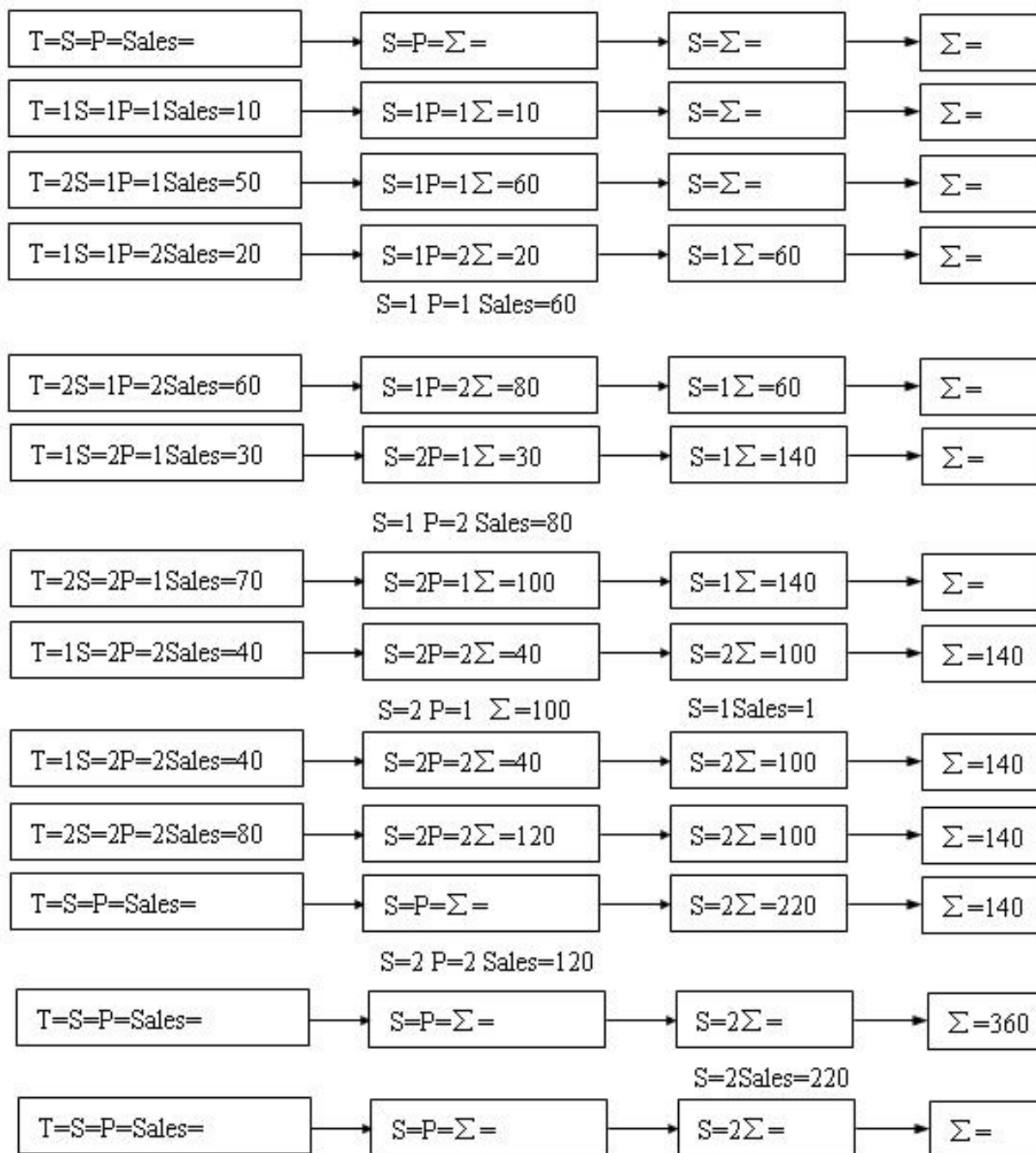
Pipe Line Algorithm

- 生成流水线的算法

$ABC \rightarrow AB \rightarrow A \rightarrow \{\}$

$BC \rightarrow B$

$AC \rightarrow C$



-
- 流水线中为每个表分配了一个存储单元，可以存放表中的一个元组。初始时，流水线中所有结点的内容为空，即最上面那条流水线。
 - 从表 (T, S, P, Sales) 读入第一条记录 (1, 1, 1, 10) 后，由于第二个结点为空，将该条记录的 S 和 P 两个维属性的值和度量的值都复制到第二个结点中，其它结点没有任何变化。

-
- 读入第二条记录 (2, 1, 1, 50) 后, 第二个结点不为空, 但是, 维属性S、P的值和第二条记录的相等, 因此, 把第二条记录的度量值和第二个结点的度量值累加 (这里假设聚集函数是sum) 。

-
- 读入第三条记录 (1, 1, 2, 20) 后, 第二个结点不为空, 其内容是 $S=1P=1\Sigma=60$, 它与当前记录在属性 **S** 和 **P** 上的值不相等, 这时, 首先要把第二个结点中的内容流动到第三个结点, 由于第三个结点为空, 就把 **S** 属性的值和度量值复制到第三个结点中。其次, 将第二个结点的内容作为一条记录输出到表 (**S**, **P**, **Sales**)。最后, 将第三条记录的 **S** 和 **P** 两个维属性的值和度量的值都复制到第二个结点中。

-
- 当处理完表 (T, S, P, Sales) 的所有记录后，流水线中有些结点不为空，这时，按照从左至右的次序，把不为空结点的内容流动到下一个结点，并组织一条记录输出到对应的表中，然后清空结点，直到所有结点都为空为止。

Pipe Line Algorithm

- 可以充分的利用排序结果
- 主要的工作量是外排序,
- 容易改写成并行算法
- 需要的内存小 (如果不考虑外排序)

Two Tier Pipe Line

- 两类维： 划分维和非划分维

Two Tier Pipe Line

- 划分维构成主流流水线

$AB \rightarrow B \rightarrow \{\}$

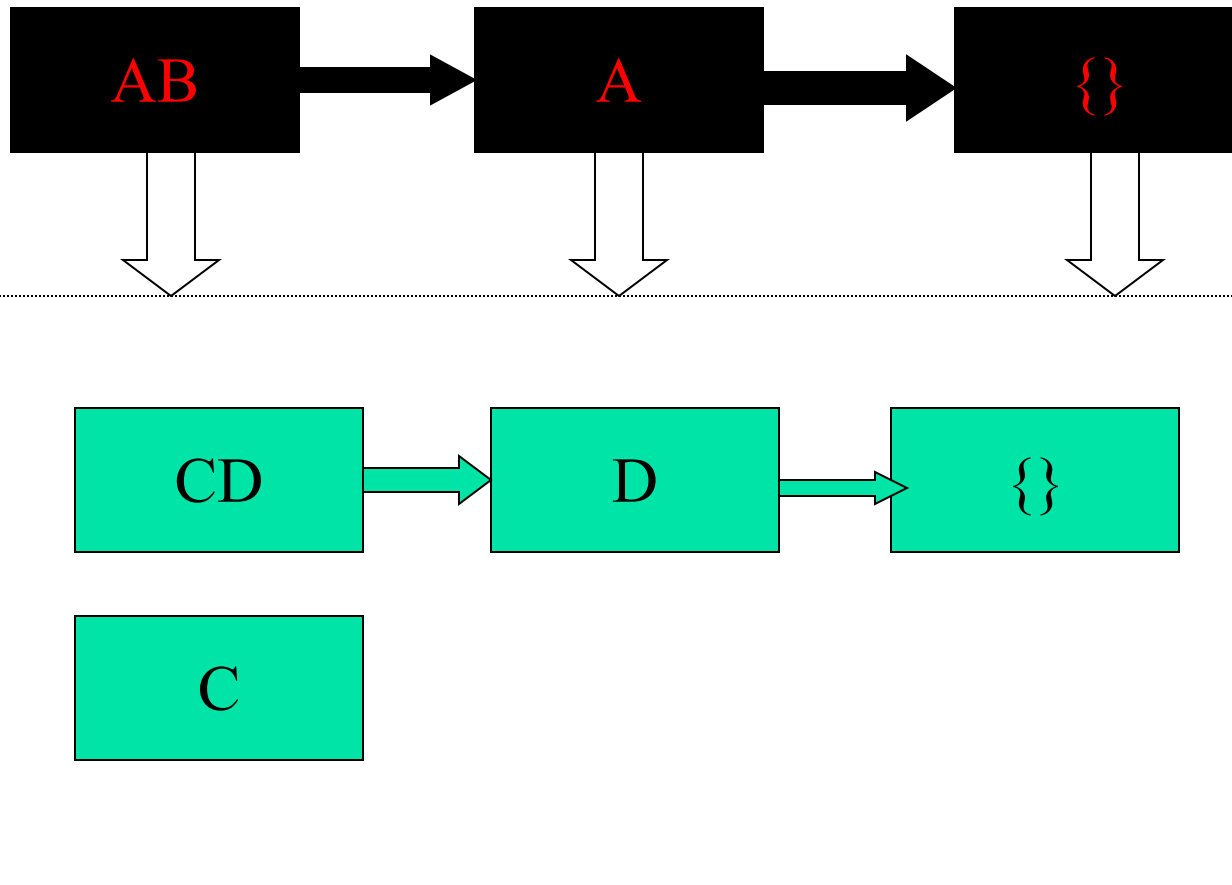
A

- 非划分维构成次流水线

$CD \rightarrow D \rightarrow \{\}$

C

Two Tier Pipe Line



Two Tier Pipe Line

